

Local Food Wastage Management System

Project Title

Local Food Wastage Management System

Objective

The objective of this project is to develop a platform that connects food providers (restaurants, grocery stores, individuals) with receivers (NGOs, community centers, individuals in need) to reduce food waste and improve food distribution efficiency.

Skills Learned

- Python programming
- SQL database creation and management
- Streamlit application development
- Data analysis and visualization
- CRUD operations implementation
- Food management and social impact applications

Dataset Description

1. **providers_data.csv**

2. **Provider_ID**: Unique identifier for each provider
3. **Name**: Name of the provider (restaurant, store, etc.)
4. **Type**: Category (Restaurant, Grocery Store, Supermarket)
5. **Address**: Physical address
6. **City**: City location
7. **Contact**: Contact information

8. **receivers_data.csv**

9. **Receiver_ID**: Unique identifier for each receiver
10. **Name**: Name of receiver (individual or organization)
11. **Type**: Category (NGO, Community Center, Individual)
12. **City**: City location
13. **Contact**: Contact details

14. **food_listings_data.csv**

15. **Food_ID**: Unique identifier for each food item

16. **Food_Name:** Name of food item
17. **Quantity:** Quantity available
18. **Expiry_Date:** Expiry date of food
19. **Provider_ID:** ID of the provider offering the food
20. **Provider_Type:** Type of provider
21. **Location:** City where food is available
22. **Food_Type:** Category of food (Vegetarian, Non-Vegetarian, Vegan)
23. **Meal_Type:** Meal type (Breakfast, Lunch, Dinner, Snacks)
24. **claims_data.csv**
25. **Claim_ID:** Unique identifier for each claim
26. **Food_ID:** ID of the food claimed
27. **Receiver_ID:** ID of the receiver
28. **Status:** Pending, Completed, or Cancelled
29. **Timestamp:** Date and time of claim

Database Setup Steps

1. Create a SQLite database named `database.db`.
2. Use Python `pandas` to read CSVs and load data into SQL tables (`Providers` , `Receivers` , `Food_Listings` , `Claims`).
3. Verify table creation and relationships:
4. `Food_Listings.Provider_ID` → `Providers.Provider_ID`
5. `Claims.Food_ID` → `Food_Listings.Food_ID`
6. `Claims.Receiver_ID` → `Receivers.Receiver_ID`
7. Ensure primary keys and constraints are correctly applied.

SQL Queries and Purpose

1. **Providers & Receivers**
 2. Count per city: Number of providers and receivers per city.
 3. Top provider type: Provider type contributing most food.
 4. Contact info by city: Helps receivers find provider contacts.
 5. Top receivers: Receivers with most food claims.
6. **Food Listings & Availability**
 7. Total quantity: Total food available.
 8. City with highest listings: Identify high availability locations.
 9. Most common food types: Trends in food types available.

10. Claims & Distribution

11. Claims per food item: Popularity of each food.
12. Provider with most successful claims: Efficiency of providers.
13. Claim status percentage: Completed, pending, canceled claims.

14. Analysis & Insights

15. Average quantity per receiver: Mean food claimed per receiver.
16. Most claimed meal type: Meal type demand trends.
17. Total donated per provider: Provider contribution overview.
18. Optional: Food nearing expiry, Receivers with pending claims.

Instructions to Run Streamlit App

1. Install dependencies:

```
pip install streamlit pandas sqlalchemy
```

1. Ensure `database.db` is located in the `database/` folder.
2. Place all CSVs in `datasets/` folder.
3. Run the Streamlit app:

```
streamlit run app/app.py
```

1. Use the sidebar filters to select city, food type, and meal type.
2. Use the CRUD form to add new food listings.
3. View visualizations and query outputs dynamically.

Summary of Insights

- **Top Providers:** Restaurants contribute the highest quantities of food.
- **High-demand Foods:** Certain meal types like Dinner and Lunch are claimed most.
- **Claim Patterns:** Most claims are completed, but some remain pending in specific cities.
- **Geographical Trends:** Cities with the highest listings correlate with high population areas.
- **Food Near Expiry:** Identified items for priority distribution to reduce waste.

This project effectively demonstrates the integration of Python, SQL, and Streamlit to manage local food wastage and provide actionable insights for better distribution and social impact.